17. GitHub Actions ve Otomasyon: AI Destekli CI/CD Boru Hatları

Her Push'ta Otomatik Test, Lint ve AI Kod Kalitesi Denetimi
Bölüm 3 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Şirketler neden 'benim makinemde çalışıyor' sözüne güvenmez ve GitHub Actions gibi CI/CD sistemleri insan denetiminin yerini nasıl alır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 CI/CD (Continuous Integration / Continuous Delivery), kodun her commit'te izole bir sunucuda otomatik test edilmesidir. - 📌 Eğer testler CI ortamında geçmiyorsa, o kod 'çalışmıyor' kabul edilir; mazeret geçerli değildir. - 📌 GitHub Actions YAML dosyalarını yapay zekaya yazdırmak, karmaşık pipeline kurulumlarını saatlerden dakikalara indirir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir otomobil fabrikasındaki montaj hattını düşünün.

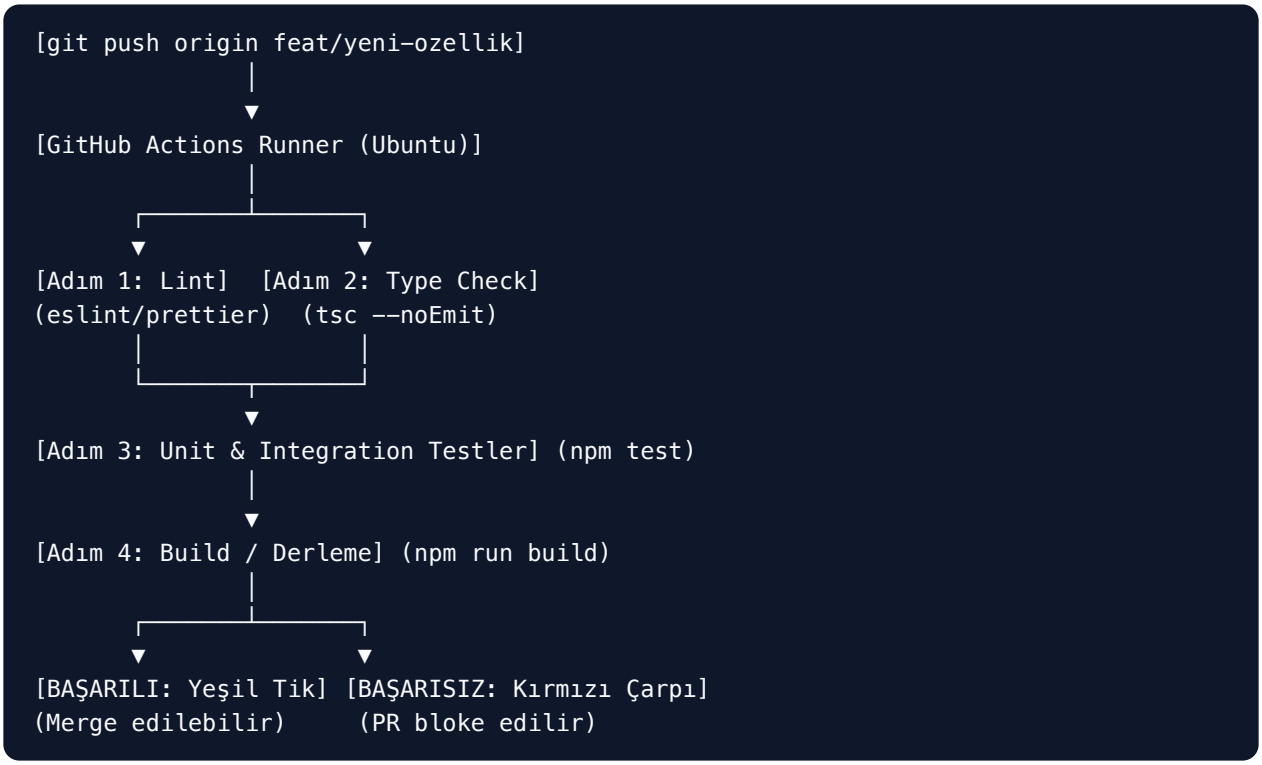
Her araba hattan çıkmadan önce otomatik bir test istasyonuna girer: Robot kollar kapıları 1000 kez çarpar, su fıskiyeleri su sızdırmazlığını kontrol eder, bilgisayarlar fren hidroliğini ölçer. Eğer araç bu testlerin tek birinden bile kalırsa hattan dışarı çıkarılamaz.

GitHub Actions işte o otomatik test robotudur!

Siz kodunuzu GitHub'a gönderdiğiniz anda, bulutta tertemiz sanal bir sunucu (Ubuntu) açılır. Projenizi kurar, linter'ı çalıştırır, tüm testleri koşar ve derler. Eğer her şey yeşil yanarsa kodunuz birleşmeye hak kazanır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

CI/CD Boru Hattı Akışı:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevOps ve CI/CD Mühendisi
Next.js ve TypeScript kullanan bir projemiz var.

## GÖREV:
GitHub Actions için `.github/workflows/ci.yml` dosyasını hazırla.
Boru hattı şu gereksinimleri karşılamalıdır:
1. `main` ve `develop` branch'lerine açılan PR'larda ve push'larda otomatik
tetiklensin.
2. Node.js 20 sürümünü kullansın ve `npm ci` ile bağımlılıkları önbelleklesin
(cache).
3. Sırasıyla şu adımları çalıştırsın:
   - Lint kontrolü (`npm run lint`)
   - TypeScript derleme kontrolü (`npx tsc --noEmit`)
   - Testler (`npm test`)
4. Herhangi bir adım çökerse pipeline anında dursun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```yaml
# Kötü CI Dosyası: Önbellekleme yok, sürüm belirtilmemiş, testleri atlıyor
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: npm install # npm ci yerine install kullanmak lockfile'ı bozar!
      - run: npm run build
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```yaml
name: Continuous Integration (CI)

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality-gate:
    name: Lint, Typecheck & Tests
    runs-on: ubuntu-latest

    steps:
      - name: 📥 Depoyu Klonla
        uses: actions/checkout@v4

      - name: 🟢 Node.js Kurulumu & Önbellek
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: 📦 Bağımlılıkları Kur (Deterministic)
        run: npm ci

      - name: 🔍 Linter & Format Kontrolü
        run: npm run lint

      - name: 🛡️ TypeScript Tip Kontrolü
        run: npx tsc --noEmit

      - name: 🧪 Unit Testleri Çalıştır
        run: npm test -- --ci --maxWorkers=2

      - name: 🏗️ Production Build Doğrulaması
        run: npm run build
```

💡 **Kod Analizi:** `npm ci` kullanılarak `package-lock.json` ile birebir uyumlu kurulum sağlanmış, `concurrency` bloğu ile aynı branch'e yeni push geldiğinde eski gereksiz CI koşuları otomatik iptal edilerek fatura maliyeti düşürülmüştür.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** CI ortamında `npm install` kullanmak.
  - **Çözüm:** CI sunucularında her zaman deterministik olan `npm ci` komutunu kullanın.
- **Tuzak:** CI kırmızı yanarken 'nasılsa yerelde çalışıyor' deyip PR'ı zorla merge etmek.
  - **Çözüm:** CI kırmızıysa kod bozuktur; hatayı CI loglarından okuyup düzeltin.
- **Tuzak:** Secret (API anahtarları) bilgilerini CI yaml dosyasına düz metin yazmak.
  - **Çözüm:** Hassas bilgileri her zaman GitHub Secrets (`${{ secrets.API_KEY }}`) üzerinden çekin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 17: GitHub Actions ile CI/CD - Kalite Kapılarını Kurmak*

- 🖥️ **Slayt 1:** CI/CD nedir ve şirketler için neden hayati önem taşır?
- 🖥️ **Slayt 2:** GitHub Actions anatomisi: Workflow, Event, Job, Step.
- 🖥️ **Slayt 3:** Deterministik kurulum: npm install vs npm ci.
- 🖥️ **Slayt 4:** Canlı Demo: Sıfırdan GitHub Actions CI pipeline kurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Reponuzda `.github/workflows/ci.yml` oluşturun. Kasıtlı olarak bir TypeScript tip hatası yapıp push edin. GitHub Actions sekmesinde build'in kırmızıya düştüğünü ve PR'ın kilitlendiğini gösterin. Hatayı düzeltip yeşil tiki alın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kişisel bir GitHub projenizde `.github/workflows/ci.yml` oluşturun ve testleri her push'ta otomatik çalıştıracak şekilde yapılandırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: CI sunucularında `npm install` yerine `npm ci` kullanılmasının temel sebebi nedir? - A) `npm ci` komutunun daha renkli çıktılar vermesi - B) `package-lock.json` dosyasını değiştirmeden, kilitli sürümleri birebir ve temiz kurarak determinizm sağlaması - C) Sadece Mac bilgisayarlarda çalışması - D) İnternet bağlantısı gerektirmemesi > **Doğru Yanıt:** B > **Açıklama:** `npm ci` mevcut lockfile'ı asla değiştirmez ve sıfırdan temiz bir kurulum yaparak beklenmedik sürüm uyuşmazlıklarını önler.

Soru 2: GitHub Actions'ta `concurrency: cancel-in-progress: true` ayarının sağladığı fayda nedir? - A) Bilgisayarı kapatması - B) Aynı branch'e yeni bir commit geldiğinde eski ve artık gereksiz olan çalışan CI işini otomatik iptal ederek sunucu süresi tasarrufu sağlaması - C) Kodları otomatik silmesi - D) Testleri yapmadan geçmesi > **Doğru Yanıt:** B > **Açıklama:** Geliştirici arka arkaya commit attığında önceki eski test koşularını iptal etmek hem kaynak tasarrufu sağlar hem de CI kuyruklarını rahatlatır.